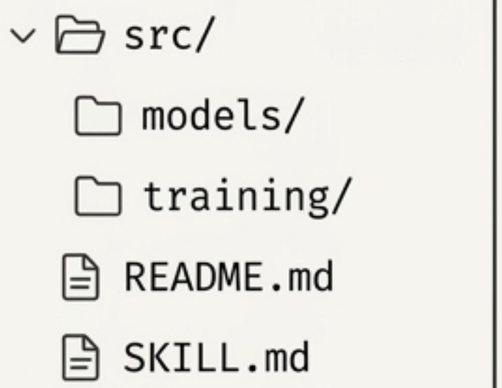


> The Architecture of Simplicity

// First-Principles AI Engineering via Andrej Karpathy

```
$ execution_mode: standalone_visual_document
$ dependencies: zero_abstractions
$ status: ready
```

NotebookLM

Stack Trace

[2024] Independent AI Educator

[2017] Tesla Director of AI (Autopilot)

[2015] OpenAI Founding Member

[2015] Stanford CS231n Architect (150 -> 750 students)

/* I am becoming seriously allergic to 500-pound websites. 0

frameworks were used to make this simple responsive website… pure HTML and CSS in two static files and that's it. */

> Mission: Train deep neural nets and make them universally understandable.

NotebookLM

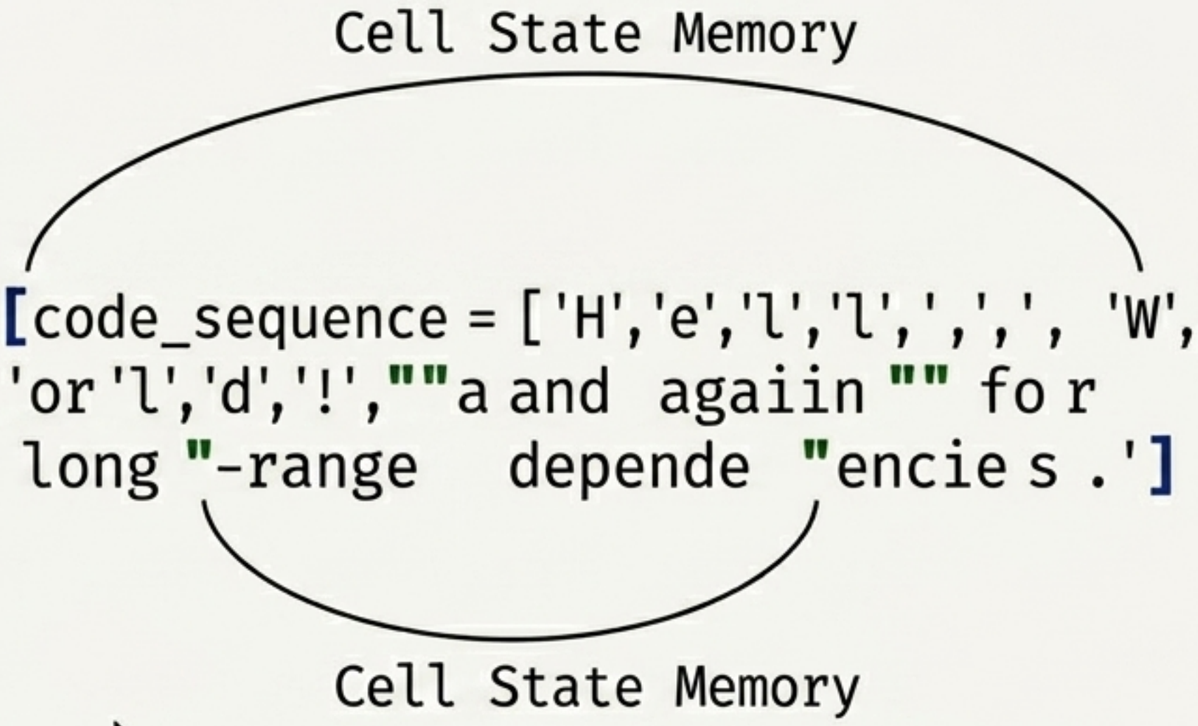
Stack Trace

[2024] Independent AI Educator

[2017] Tesla Director of AI (Autopilot)

[2015] OpenAI Founding Member

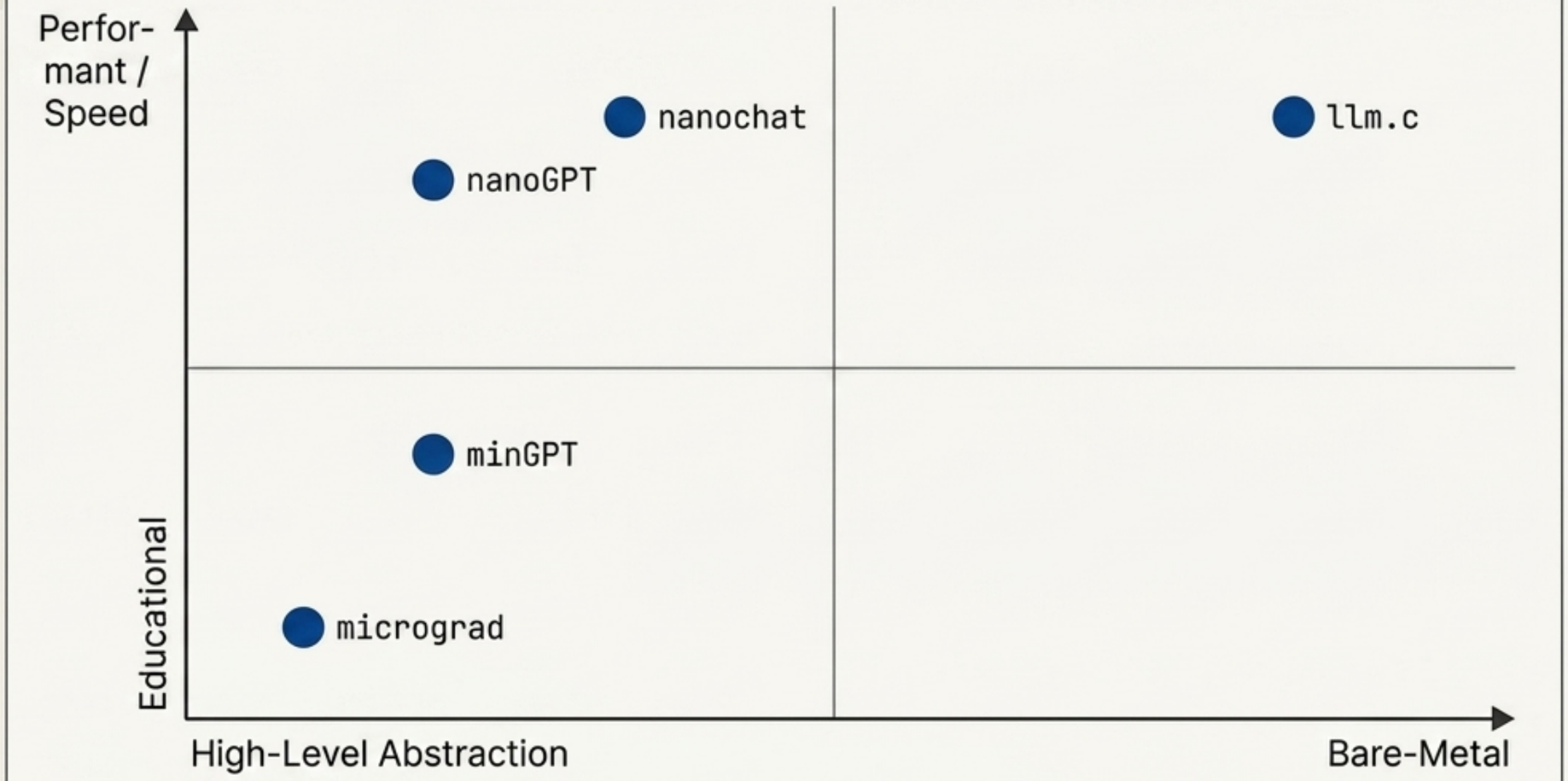
[2015] Stanford CS231n Architect (150 -> 750 students)



Stripping complexity to find interpretable cells: Tracking line lengths, quotes, and brackets at the character level.

> paper_id: arXiv:1506.02078

NotebookLM



Terminal

```
> query: select_optimal_repo() // Output: Depends on user goal (learn vs. deploy).
```

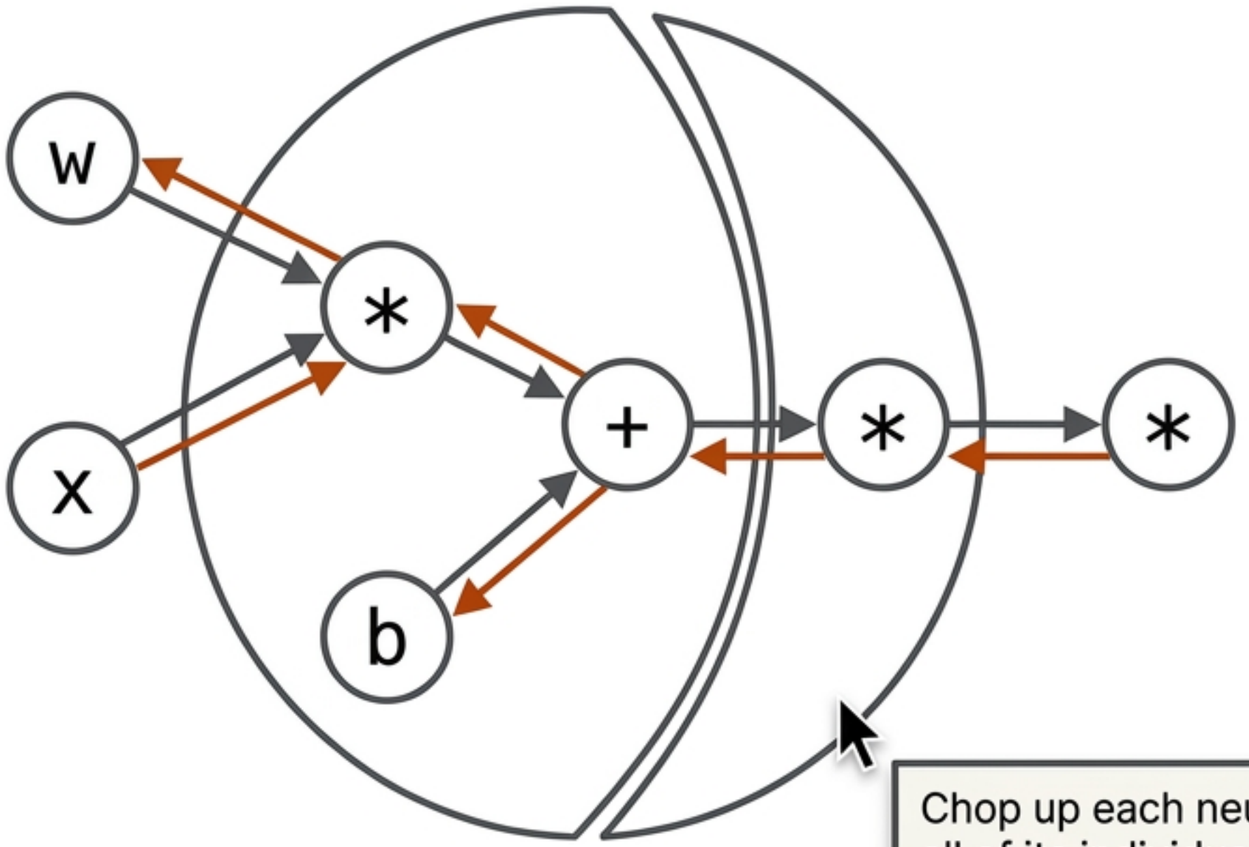
NotebookLM

Repo: karpathy/micrograd

Lines: ~100 (Autograd) / ~50 (NN library)

Dependencies: 0

The Scalar DAG



NotebookLM

minGPT

The Educator

- **Key Stat:** ~300 lines of PyTorch.
- **Focus:** Clean, interpretable, hackable implementation of OpenAI's Generative Pretrained Transformer.

nanoGPT

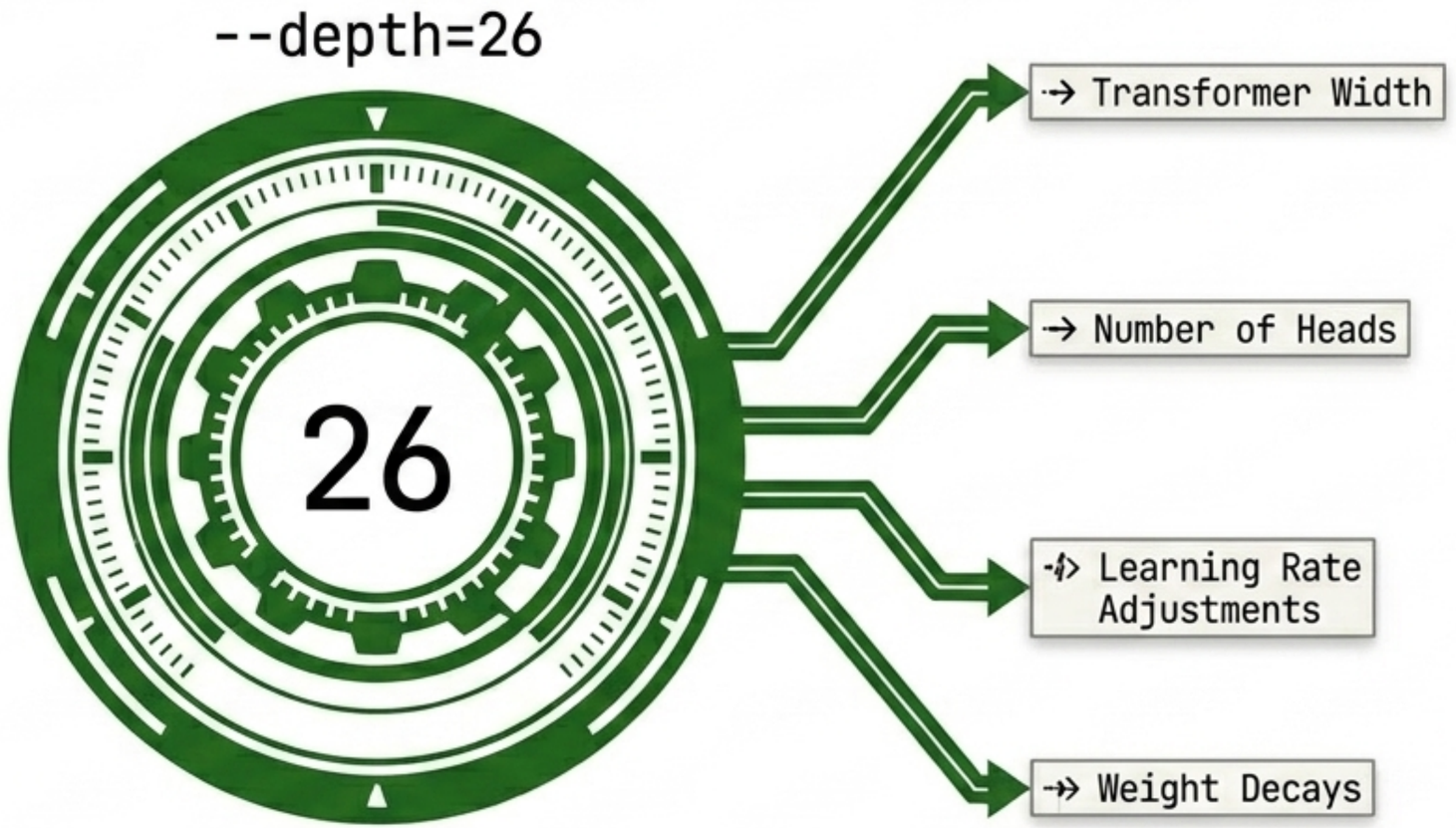
The Practitioner

- **Key Stat:** Reproduces GPT-2 (124M) in ~4 days on a single 8xA100 node.
- **Focus:** Prioritizes teeth over education. Achieves ~2.85 loss on OpenWebText.

> transition: minGPT -> nanoGPT
> result: retain readability, maximize hardware utilization.

NotebookLM

The nanoGPT Parameter

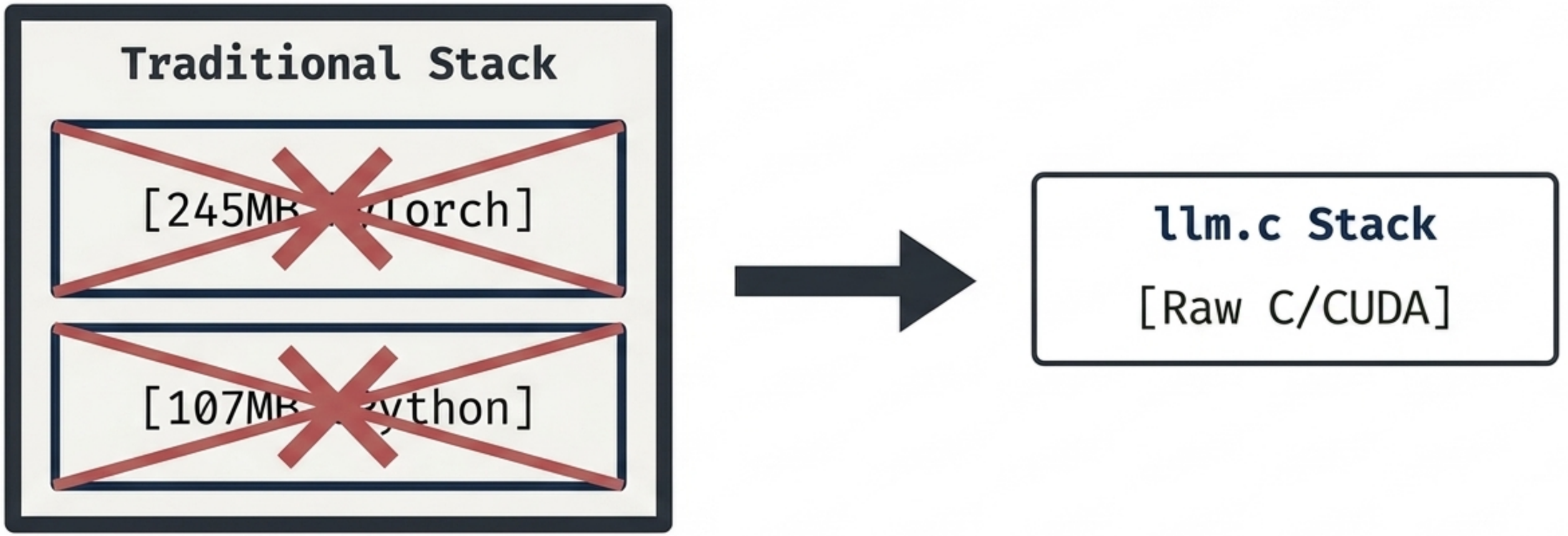


Time-to-GPT-2 Leaderboard

2019	OpenAI: $43,000 / 168 hours
2026 ✓	nanochat: <$100 / 1.65 hours (DCLM CORE 0.2626)

> The single complexity dial: calculate all optimal hyperparameters automatically based on layer depth.

NotebookLM



Performance: **7% faster** than **PyTorch Nightly**.

Codebase: **~1,000 lines of clean C** (CPU fp32 reference).

> I'd like this repo to only maintain C and CUDA code.

NotebookLM

Issue Tracker: LLM Coding Anti-Patterns

[ERR 01] **Silent Assumptions**: Making wrong assumptions without seeking clarification.

[ERR 02] **Bloated Abstractions**: Writing a 1000-line construction when 100 would do.

[ERR 03] **Orthogonal Edits**: Changing code they don't understand as a side effect.

[ERR 04] **Weak Verification**: Lacking success criteria to self-correct.

> **fix_available**: Applying the 4 Karpathy Principles...

NotebookLM

Principle 1: Think Before Coding

Anti-Pattern

Silently guessing ambiguous requirements.

Karpathy Rule

Surface tradeoffs explicitly. Stop and ask when confused.

Principle 2: Simplicity First

Anti-Pattern

Flexibility that wasn't requested.

Karpathy Rule

Minimum code to solve the problem. If 200 lines could be 50, rewrite it.

> test_condition: Would a senior engineer say this is overcomplicated?

NotebookLM

Principle 3: Surgical Changes

Anti-Pattern

Drive-by refactoring of adjacent, unbroken code.

Karpathy Rule

Touch only what you must. Match existing style perfectly.

Principle 4: Goal-Driven Execution

Anti-Pattern

Instructing the model to 'Fix the bug.'

Karpathy Rule

Instruct the model to 'Write a test that reproduces it, then make it pass.' Loop until verified.

> insight: LLMs excel at looping independently if given strict verification criteria.

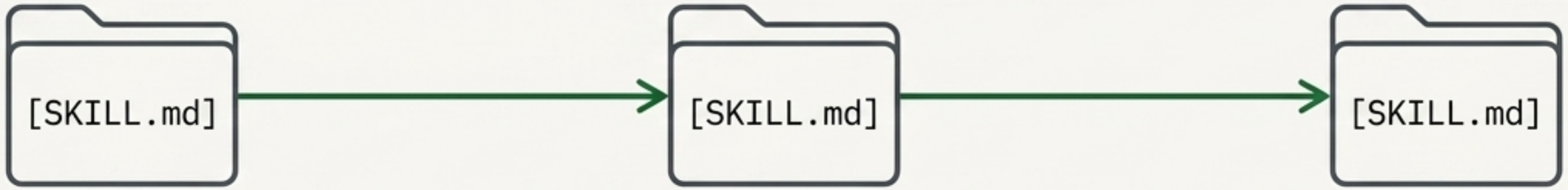
NotebookLM

The Old Way: The Monolith



The model is the runtime. The folder is the program.
Two skills written by two different people,
never tested together, just compose.

The Unix Philosophy: The Skill Primitive



NotebookLM

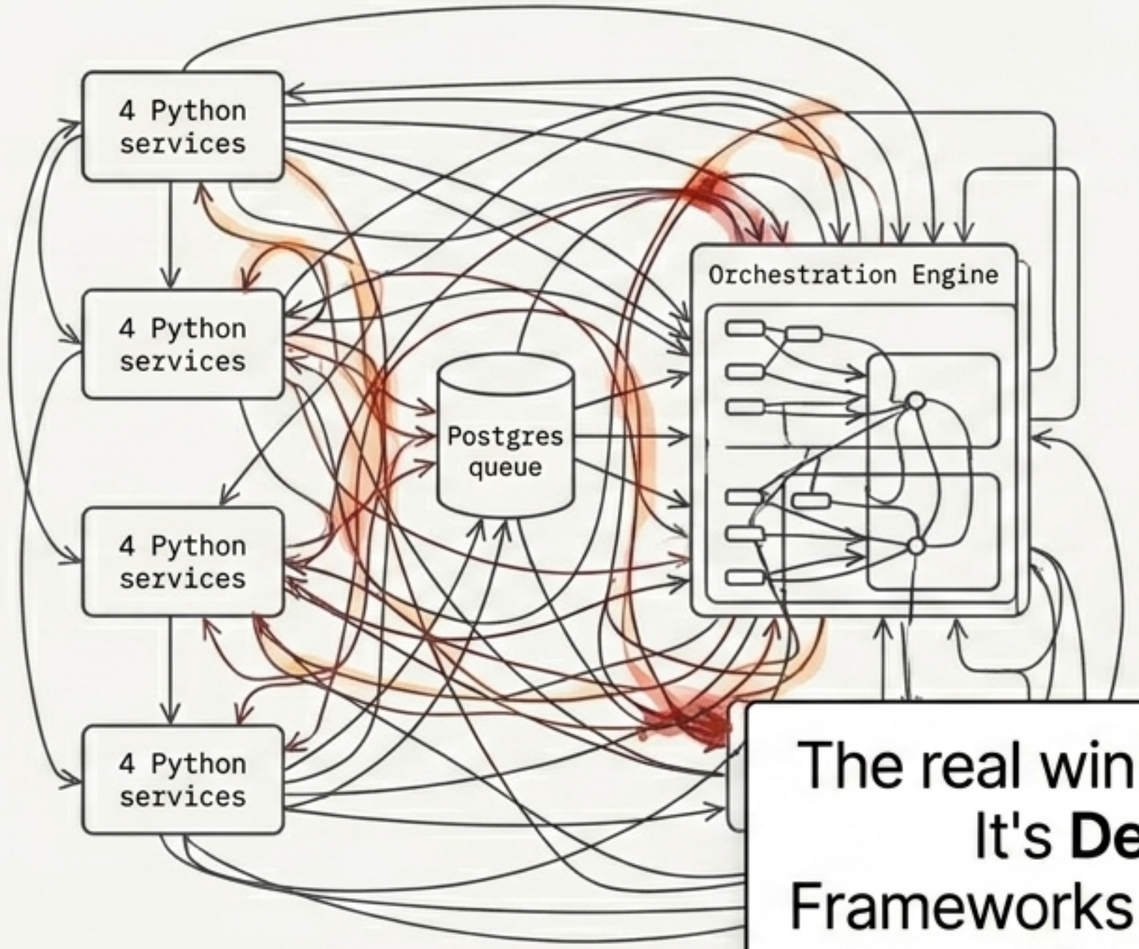
Agent Frameworks vs. The Skill Primitive

	Traditional Wrappers (LangChain/MCP)	**Claude Skills (SKILL.md)**
System Prompt	Bloated	Stays clean (loads on match)
Integration Code	Heavy orchestration	Zero integration required
Nature of Unit	Tool wrapper function	Self-contained folder
Execution Engine	Python State Machine	The LLM Itself

> conclusion: Stop treating the model as a function caller.

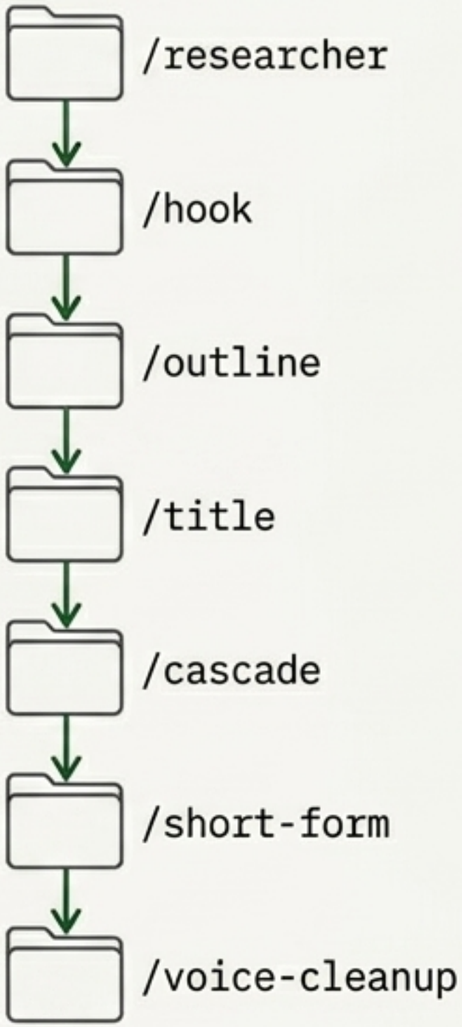
NotebookLM

Before



~2,400 lines of code.

After

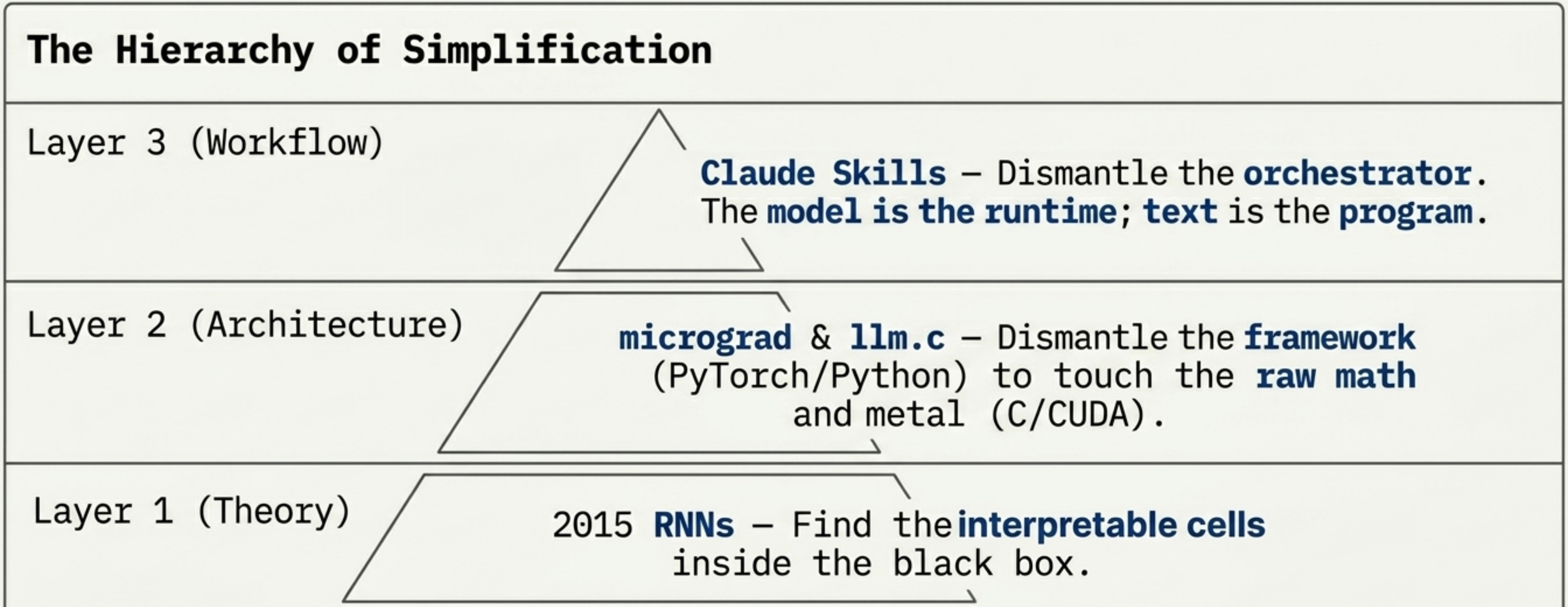


~600 lines of code. (75% less)

The real win isn't just speed.
It's **Deletability**.
Frameworks punish deletion.
Skills don't.

NotebookLM

The Hierarchy of Simplification



True AI mastery isn't about building more complex abstractions. It's about **systematically dismantling** them until only the **primitive truth** remains.

NotebookLM

> // The Karpathy Way:

> **Write** skills like **documentation**, not **code**.

> **Start** from **scratch**.

> **Strip** the **bloat**.

> Let the **model** be the **runtime**.

```
$ process_terminated_successfully
$
```

NotebookLM